

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)
ORGANISATION OF ISLAMIC COOPERATION (OIC)
Department of Computer Science and Engineering (CSE)

Final Report

SWE 4404: Software Project Lab-II

Project Title

FLUX

A Visual Data Pipeline Builder

Team 5

Mahmudul Hossain Samir (230042101)
Ifham Bin Hossain(230042112)
Shaikh Muhaiminul Hasan(230042113)

Supervisor

Samnun Azfar
Junior Lecturer
Department of CSE, IUT

Nov 1, 2026

1. Executive Summary---------- 2
2. Problem Domain and Background---------- 3
3. Literature and Existing Solutions----------5
4. Requirements(SRS Session)---------- 7
5. System Design(Design Session)---------- 11
6. Implementation(Implementation Session)---------- 16
7. Tools, Version Control, Testing, and CI/CD---------- 23
8. Professional, Ethical, and Legal Considerations---------- 25
9. Project Management and Feasibility---------- 27
10. Teamwork and Individual Contributions---------- 30
11. Results, Limitations, and Future Work----------33
12. Glossary and References---------- 36

1. Executive Summary

Every day, millions of business professionals, accountants, educators, and students work with tables of information stored in spreadsheets such as Microsoft Excel or comma-separated files. While spreadsheets are convenient for viewing small tables, performing multi-step analysis such as removing duplicate records, filtering out invalid entries, combining two separate tables, and calculating regional summaries quickly becomes frustrating and error-prone especially for beginner and intermediate level users. To accomplish these tasks in a traditional spreadsheet, a person must write long, deeply nested formulas inside tiny grid cells. When a column moves or new monthly data arrives, these formulas frequently break, displaying cryptic error codes without explaining where or why the mistake happened. Conversely, programming tools like Python or Pandas can automate data cleaning reliably, but they require years of coding experience that most everyday spreadsheet users do not possess.

FLUX (A Visual Data Pipeline Builder) solves this problem by turning spreadsheet analysis into an intuitive, visual flowchart inside a standard web browser. Instead of writing complex formulas or computer code, users build an analytical workflow by dragging and dropping self-contained building blocks such as *Read CSV, Filter Rows, Remove Duplicates, Join, and Group By & Aggregate* onto an open canvas and connecting them with arrows. As data moves from left to right across these connected blocks, FLUX immediately shows a live preview table at every step and users can also see visuals through visual chart based nodes like Bar Chart, Line Chart or Pie Chart. Users can also click a “Replay” button to watch a step-by-step animation of how their raw file is transformed into their desired result so the option for seeing intermediary results at every step becomes possible through FLUX.

A standout capability of FLUX is its two-way bridge with Microsoft Excel. If a user builds a visual workflow on the FLUX canvas, the system automatically writes the equivalent Excel formula (such as FILTER, SORT, UNIQUE, XLOOKUP, or SUMIF). Furthermore, registered Users can save their visual workflows as reusable templates: when next month’s sales or budget file arrives, they simply upload the new file and run the entire analysis with a single click. Guests can also use the full analytical workspace immediately without creating an

account, they can save their templates through Exporting the Importing the templates into their equivalent JSON files.

Final Outcome: FLUX has been fully implemented, tested, and verified against all 23 Functional Requirements specified for this project. The platform provides 18 modular data-processing blocks across five categories, and supports both .csv and multi-sheet .xlsx files.

Self-Assessment CO6: Communication to a Non-Technical Audience
Question (CO6): Can you communicate the project effectively to a non-technical audience without relying on unexplained jargon?
Defense & Evidence:
• **Jargon-Free Framing:** The Executive Summary above intentionally avoids software engineering acronyms (such as DAG, AST, ORM, JSONB, or JWT) and explains the core value proposition through everyday metaphors: “building blocks,” “visual flowcharts,” and “connecting arrows.”
• **Stakeholder Validation:** During our stakeholder interview with a mid-level data analyst in Dhaka and our SRS presentation, non-technical reviewers immediately grasped the workflow through our descriptions and rough diagrams.

2. Problem Domain and Background

2.1 Application Domain and Problem Statement

FLUX operates within the domain of End-User Data Preparation, Self-Service Business Intelligence (BI), and Visual Dataflow Programming. Organizations across finance, retail, healthcare, education, and public administration generate tabular datasets stored as Comma-Separated Values (.csv) and Microsoft Excel Open XML (.xlsx) workbooks. Before any meaningful decision-making or charting can occur, raw data must undergo Extract-Transform-Load (ETL) preparation: type coercion, null imputation, deduplication, relational joining, filtering, and aggregation.

Currently, domain users face a sharp dichotomy:

1. **Coordinate-Bound Spreadsheets (e.g., Microsoft Excel):** Data and transformation logic are tightly coupled to physical cell coordinates (e.g., A2:G500). Multi-stage transformations require nesting functions inside a single cell (e.g., =SORT(FILTER(UNIQUE(A2:D100), B2:B100="Active"), 3, -1)). Intermediate states are invisible unless the user manually creates helper columns or scratch sheets. If an upstream column is inserted or a data type mismatches on row 412, the entire formula evaluates to an opaque #VALUE!, #REF!, or #N/A token with zero lineage tracing.
2. **Programmatic Dataframes (e.g., Python Pandas, R Tidyverse, SQL):** While code-based pipelines separate transformation logic from raw data and offer strong reproducibility, they impose a steep syntactic and environmental barrier (virtual environments, IDE setup, dataframe indexing semantics) that excludes non-programmer analysts.

2.2 Target Users and Why It Matters

• **Business and Financial Analysts**: Professionals who prepare recurring monthly reconciliations, budget rollups, and KPI summaries and need reusable templates plus native Excel formula output.
• **Data Engineers and Reviewers**: Technical staff who prototype lightweight ETL pipelines, join multi-source tables, and audit transformation lineage before production deployment.
• **Students, Educators, and Researchers:** Academic users who clean survey or experimental datasets and use visual step-by-step animations to understand relational data operations without writing code.
Solving this problem matters because studies of enterprise spreadsheet usage consistently show that human formula errors and copy-paste mistakes in manual spreadsheets cause costly financial and operational miscalculations, while engineering bottlenecks delay routine reporting.
Self-Assessment CO1: Fundamental & Specialist Software Engineering Knowledge
Question (CO1): Can you explain which fundamental programming, data-structure, database, and design concepts underlie the project problem?

Defense & Evidence:
1. **Graph Theory & Data Structures (Directed Acyclic Graphs & Queues):** A visual data pipeline is mathematically a Directed Acyclic Graph G = (V, E), where vertices V are transformation nodes and directed edges E represent data dependencies. Executing the pipeline requires a valid topological ordering so that no node runs before its upstream dependencies. In Backend/app/engine/executor.py (lines 176–214), we represent the graph using adjacency lists (children: dict[str, list[str]], parents:dict[str, list[str]]) and an in-degree hash map (in_degree: dict[str, int]), and traverse it in O(|V | + |E|) time using a double-ended queue (collections.deque) via Kahn's algorithm.

2. Recursion & Stack-Based Tokenization: Bidirectional Excel formula translation requires recursive tree traversal for forward composition (FormulaEngine._compose(), lines 62–94 of formula_engine.py) and depth-tracked lexical scanning (_split_top_level_args(), lines 178–213) to split nested function arguments without breaking on commas inside inner parentheses or string literals.

3. Relational & Semi-Structured Database Concepts: Persistent entities require ACID transactions, foreign-key referential integrity (user.id →pipeline_template.owner_id → execution_log.template_id), domain check constraints (ck_execution_log_status), and semi-structured document storage (JSONB in PostgreSQL with a dialect variant fallback to JSON in SQLite) to store arbitrary graph topologies without rigid join tables for every canvas node (Backend/app/models/pipeline_template.py).

4. Object-Oriented Design Principles (SOLID & Design Patterns):
• **Strategy & Template Method Pattern:** The abstract base class NodePlugin(ABC) (Backend/app/nodes/base.py) defines a polymorphic contract (get_config_schema, infer_output_schema, execute, to_excel_formula, compute_row_errors), allowing DAGExecutor to execute all 18 node types uniformly without if/elif type-switching chains (Open/Closed Principle).
• **Registry / Singleton Pattern:** NodeRegistry (Backend/app/engine/registry.py) maintains an O(1) dictionary lookup of registered plugin instances and exposes metadata dynamically to the React

frontend via GET /api/nodes.

Question (CO1): Can you identify the relevant specialist knowledge used by the project and explain why it is necessary for this application domain?

Defense & Evidence:

1. Vectorized Columnar DataFrame Algebra & Static Schema Inference: Unlike general CRUD web applications that manipulate single database rows, a data pipeline engine must execute relational algebra operations (selections, projections, equi-joins, grouped aggregations, and pivot cross-tabulations) over tens of thousands of rows with sub-second latency. Specialist knowledge of vectorized Pandas series operations (Backend/app/dataframe/frame.py), two-stage heuristic type and date inference (Backend/app/dataframe/inference.py), and pre-execution schema propagation (infer_output_schema()) is essential to catch column-missing or type-mismatch errors before expensive computation and without leaking IEEE-754 NaN values into JSON responses.

2. Spreadsheet Formula Grammar & Dynamic Array Semantics: Translating between visual dataflow graphs and spreadsheet formulas requires specialist knowledge of Excel's calculation engine—specifically distinguishing legacy aggregation functions (SUMIF, COUNTIF, AVERAGEIF, VLOOKUP, compatible with Excel 2016+) from modern spilled Dynamic Array functions (FILTER, SORT, UNIQUE, XLOOKUP, CHOOSECOLS, requiring Excel 365). This specialist logic is implemented in Backend/app/engine/formula_engine.py (lines 149–158) and individual node plugins.

3. Multi-Encoding Binary Stream Parsing: Real-world spreadsheets uploaded by analysts frequently use non-UTF-8 encodings (e.g., Windows-1252, UTF-16, Latin-1), non-standard delimiters, or multi-sheet OpenXML zip containers. Our ingestion layer (Backend/app/csv_io/parser.py and xlsx_io.py) applies charset-normalizer fallback detection, delimiter sniffing, and raw-byte caching (DataReferenceCache.get_raw()) to re-parse secondary Excel worksheets on demand.

3. Literature and Existing Solutions

During the problem formulation and SRS phase, the team reviewed commercial platforms, open-source tools, and foundational academic literature on visual dataflow languages and spreadsheet error taxonomy.

3.1 Review of Existing Applications and Technical Sources

1. Microsoft Excel & Power Query: Excel [1] is the ubiquitous industry standard for tabular analysis. While Excel 365 introduced Dynamic Array functions (FILTER, SORT, UNIQUE, XLOOKUP), complex pipelines still require nesting functions inside a single formula bar. Power Query (Get & Transform) provides a step list alongside an M-language script, but it uses a linear list rather than a free-form branching canvas, requires a desktop installation for full capabilities, and cannot translate transformation chains back into standard cell formulas.

2. Programmatic Data Science Libraries (Python Pandas & Jupyter Notebooks): McKinney's Pandas library [2] provides expressive relational and statistical operations over in-memory DataFrame structures. However, as documented in human-computer interaction studies on data wrangling [3], non-programmers face high cognitive load when translating mental data models into imperative Python syntax, managing virtual environments, and repeatedly invoking df.head() to inspect intermediate state.

3. Enterprise Visual ETL Platforms (Alteryx Designer & KNIME Analytics Platform): Alteryx and KNIME [4] pioneer node-and-wire visual dataflow canvases for enterprise ETL. However, both are heavyweight desktop suites with steep licensing costs (thousands of dollars per seat annually for Alteryx) or hundreds of complex machine-learning nodes that overwhelm casual spreadsheet users.

4. Research on Spreadsheet Errors and Visual Dataflow: Panko's landmark empirical studies on spreadsheet errors [5] demonstrate that nearly 90% of large spreadsheets contain nontrivial formula errors due to hidden cell dependencies and lack of modular inspection. Johnston et al.'s survey on Dataflow Programming Languages [6] establishes that representing transformations as explicit nodes in a Directed Acyclic Graph (DAG) eliminates hidden side effects and localizes faults

3.2 How FLUX Builds On and Differs from Prior Solutions

Table 1 contrasts FLUX against existing solutions across eight core dimensions identified in our SRS and project documentation.

Capability	MS Excel	Python/Pandas	Alteryx/KNIME	FLUX
Learning Curve	Low initially; steep for nested formulas	Steep(Requires Python Syntax)	Moderate to High(Heavyweight desktop required)	Low(Zero-install browser canvas)
Logic Visibility	Hidden inside grid cells	Abstract Source Code	Visual Node Graph	Visual DAG Flowchart
Intermediate Data Previews	Manual helper columns required	Manual df.head() calls	Requires Manual Run and Inspector Window	Automatic Live Preview at every node
Error Diagnostics	Generic cell tokens (#REF!, #N/A)	Multi-line stack tracebacks	Log console messages	Red-glow node + row highlight
Step-by-Step Replay	Formula Evaluator dialog(Text Only)	None(External Debuggers)	None	Animated visual canvas replay
Template Reusability	Fragile(Coordinate Dependent)	High(via scripts)	High(workflow files)	High(1 Click Cloud Template+File

				Swap)

Table 1: Comparison of FLUX Against Existing Data Analysis Solutions

Self-Assessment CO2: Prior Solutions & Technical Sources in SRS
Question (CO2): Does your SRS discuss prior solutions or credible technical sources and explain how your problem statement builds on or differs from them?

Defense & Evidence:
• **Grounding in Empirical & Technical Sources:** In our Project Proposal (Sections 1–3, pages 2–4), README comparison analysis (lines 17–61), and SRS specification, we analyzed the failure modes of Microsoft Excel (coordinate fragility, nested VLOOKUP/IF chains, opaque debugging) and Python Pandas (syntactic barrier for non-programmers), complemented by a real-life stakeholder interview with a financial analyst in Dhaka
• **Clear Differentiation:** FLUX builds upon the visual node-and-wire paradigm of KNIME/Alteryx and the vectorized relational algebra of Pandas [2], but differentiates itself by introducing (1) instant 100-row paginated node previews without helper columns (FR_15), (2) step-by-step animated dataflow replay (FR_21) in a lightweight, zero-installation web application.

4. Requirements(SRS Session)

4.1 Stakeholder Identification

Requirements engineering combined domain analysis with a real-life stakeholder interview (a mid-level data analyst at a private firm in Dhaka who relies on Excel for monthly budget reporting) and role-based persona modeling. Nine distinct stakeholder roles were identified in the SRS:

1. New User: An unauthenticated visitor who wants to register a personal account to persist workflows.
2. Registered User: An authenticated user who logs in to save, browse, update, and reload reusable pipeline templates.
3. Guest User: An unauthenticated user (introduced following SRS panel feedback) who has full access to upload files, build pipelines, preview data, translate formulas, and export results without logging in, except for saving persistent templates.
4. Data Analyst: A domain user focused on uploading .csv/.xlsx files, filtering, sorting, deduplicating, imputing missing values, grouping/aggregating, executing pipelines, and exporting processed datasets.
5. Pipeline Builder: A workflow author who drags modular nodes onto the visual canvas and projects column subsets.
6. Data Engineer: A structurally focused user who wires node handles, configures multi-input relational joins (INNER, LEFT, RIGHT, OUTER), and relies on cycle prevention to guarantee valid DAG execution.
7. Analytics User: A power user who configures node-specific parameters and defines calculated columns from mathematical expressions.
8. Reviewer / Auditor: A stakeholder who inspects intermediate node previews, locates

faulty nodes via visual error highlighting, and plays step-by-step dataflow animations to verify pipeline correctness.
9. Spreadsheet User: An Excel practitioner who generates equivalent Excel formulas from visual nodes or pastes legacy Excel formulas to visualize them as a node graph.

4.2 Functional Requirements and Traceability Matrix

The SRS specifies 23 Functional Requirements (FR_1–FR_23) decomposed into 25 User Stories(US_1–US_25). Table 2 provides the complete traceability matrix mapping each functional requirement and user story to its stakeholder need, priority, and implementing backend/frontend modules.

FR ID	US ID	Stakeholder Role	Requirement & Stakeholder Need Summary	Priority	Implementing Modules
FR_1	US_1	New User	Register a new account with email and password to save and access pipelines.	Medium	app/auth/manager.py, AuthModal.jsx
FR_2	US_2	Registered User	Log in with credentials and receive a JWT token to access saved templates.	Medium	app/auth/backend.py, useAuth.js
FR_3	US_3	Registered User	Display a list of the user's saved pipeline templates with name and modified date.	Medium	routers/templates.py,TemplateBrowser.jsx
FR_4	US_4	Data Analyst	Import source data by uploading standard .csv files and display columns/row count	High	csv_io/parser.py, nodes/source/read_csv.py
FR_4	US_5	Data Analyst	Import source data by uploading .xlsx Excel workbooks (including sheet selection)	Medium	csv_io/xlsx_io.py, nodes/source/read_xlsx.py
FR_5	US_6	Pipeline Builder	Provide a drag-and-drop canvas where users add pipeline nodes from a sidebar	High	ReactFlowCanvas.jsx, NodeSidebar.jsx
FR_6	US_7	Data Engineer	Connect node output/input handles with edges to define data transformation order.	High	ReactFlowCanvas.jsx, edgePorts.js
FR_7	US_8	Analytics User	Configure node-specific parameters via a dynamic configuration panel	High	ConfigPanel.jsx, FieldRenderer.jsx
FR_8	US_9	Data Analyst	Filter rows based on column conditions (=, !=, >, <, >=, <=, CONTAINS, etc.).	High	nodes/transform/filter_node.py
FR_9	US_10	Data Analyst	Sort rows by a selected column in ascending or descending direction	Medium	nodes/transform/sort_node.py
FR_10	US_11	Pipeline Builder	Select a subset of columns to	Medium	select_columns.py,

			retain and drop unnecessary fields (plus rename/limit).		rename_column.py, limit_rows.py
FR_11	US_12	Data Analyst	Remove duplicate rows across selected key columns or entire records.	Medium	nodes/transform/remove_duplicates.py
FR_12	US_13	Data Analyst	Fill missing (null) values using constant, mean, median, or forward-fill strategies	Low	nodes/transform/fill_missing.py
FR_13	US_14	Analytics User	Add a new calculated column computed from a mathematical expression across columns	High	nodes/transform/add_calculated_column.py
FR_14	US_15	Data Analyst	Group rows by column(s) and apply aggregate functions (SUM, COUNT, AVERAGE, MIN, MAX) or pivot tables.	High	group_by_aggregate.py, pivot.py
FR_14	US_16	Data Engineer	Join two data sources on a common key (INNER, LEFT, RIGHT, OUTER).	High	nodes/join/join_node.py
FR_15	US_17	Reviewer	Preview intermediate output data (paginated 100-row table and charts) for any selected node.	High	DataPreviewTable.jsx, ChartPreview.jsx
FR_16	US_18	Data Engineer	Prevent cyclic connections between nodes so pipelines always form a valid DAG	High	executor.py (kahn_topological_sort), routers/pipeline.py
FR_17	US_19	Reviewer	Visually highlight faulty nodes with a red glow indicator and flag affected row indices.	High	FluxNode.jsx, error_messages.py
FR_18	US_20	Spreadsheet User	Generate the equivalent Excel formula	Medium	FormulaEngine.generate(), FormulaPanel.jsx
FR_20	US_22	Data Analyst	Execute the pipeline end-to-end in topological order and display final output or error	High	DAGExecutor.execute(), routers/pipeline.py
FR_21	US_23	Reviewer	Play, pause, and step through an animated replay of data flowing across pipeline nodes.	Low	ReplayControls.jsx, pipelineStore.js
FR_22	US_24	Registered User	Save the current pipeline as a named template and reload it onto the canvas later.	Medium	models/pipeline_template.py, useTemplates.js
FR_23	US_25	Data Analyst	Export the full processed pipeline output as a downloadable .csv or .xlxs file	High	DAGExecutor.execute_for_export(), routers/pipeline.py

Table 2: Complete SRS Functional Requirements and Stakeholder Traceability Table

4.3 Non Functional Requirements(NFRs)

Table 3 details the Non-Functional Requirements distinguished from functional behaviors and traced to stakeholder quality needs.

NFR _ID	Category	Specification & Metric	Stakeholder Need & Implementation

NFR_1	Performance and Latency	Update live node previews with minimal perceptible latency; 300ms frontend debounce and paginated 100-row preview payloads.	Analysts need interactive feedback (FluxDataFrame.preview(100)).
NFR_2	Scalability & Throughput	Process tabular datasets up to 50,000 rows (and up to 10 MB upload size) without UI freezing or memory exhaustion	Financial analysts working with multi-month ledgers(_MAX_UPLOAD_BYTES = 10MB).
NFR_3	Usability & Accessibility	Zero coding required; plain-English error messages with actionable fix suggestions (suggest_fix()).	Non-programmer domain users and students.
NFR_4	Security & Rate Limiting	Password hashing via Argon2/Bcrypt, JWT Bearer tokens (HS256), and tiered rate limiting (30 req/min guest, 60 req/min auth).	Protection against brute-force and DoS (app/rate_limit.py)
NFR_5	Privacy & Ephemeral Data	Uploaded row-level data is stored only in an in-memory TTL cache (2 hours) and never persisted to disk/DB	Corporate confidentiality for financial/HR spreadsheets (app/data_cache.py)
NFR_6	Portability & Extensibility	Cross-dialect ORM (GUID, JSONB with SQLite JSON fallback) and modular NodePlugin architecture.	Zero-config local dev + production PostgreSQL compatibility.

Table 3: Non-Functional Requirements and Verification Metrics

4.5 SRS Session Feedback and Resulting Changes

1. Introduction of the Guest User Role (SRS Session Feedback): During the initial SRS presentation, the evaluation panel noted that forcing mandatory account registration before allowing a user to test the canvas created unnecessary onboarding friction. Action Taken: We introduced the Guest User actor in the Level 0 Context Diagram (doc/design.pdf, page 3) and implemented optional JWT injection (current_user_optional in Backend/app/auth/dependenGuests have full access to upload files, build pipelines, preview data, translate formulas, and export results, with the sole exception that saving persistent

templates requires authentication (current_active_user).

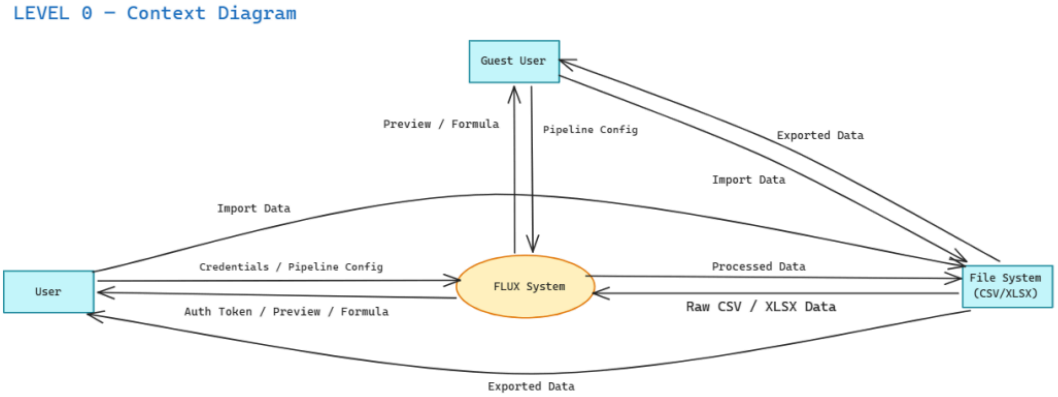


Figure 1: Level 0 – Context Diagram

2. Deferral of Real-Time Multi-User WebSockets (Proposal/SRS Feedback): The panel suggested exploring WebSocket-based collaborative editing. Following consultation with our supervisor (doc/Proposal Report.pdf, Section 9), we prioritized core DAG execution, 18 deterministic node plugins, and bidirectional formula compilation first, scheduling WebSocket collaboration as a post-SPL-II extension.

Self-Assessment CO2: Distinguishing & Tracing Functional and Non-Functional Requirements
Question (CO2): Are functional and non-functional requirements distinguished and traceable to stakeholder needs?

Defense & Evidence:
• **Explicit Separation:** Functional requirements (FR_1–FR_23, defining what transformations, formula translations, and template operations the system performs) are explicitly separated from Non-Functional Requirements (NFR_1–NFR_6, defining how well the system performs regarding latency, 50,000-row throughput, ephemeral privacy, and security).
• **Bidirectional Traceability:** Every requirement in Table 2 maps to a specific user story (US_1–US_25), a precise stakeholder role (e.g., Financial Data Analyst, Reviewer, Spreadsheet User), acceptance criteria in doc/srs.pdf, and concrete source files in code/Backend and code/Frontend.

5. System Design(Design Session)

5.1 Architectural Overview: From 3-Layer Conceptual Model to 4-Tier Implementation

In the Design Document (doc/design.pdf, page 8), FLUX is structured around a clean ThreeLayered Architecture separating the Presentation Layer, Application Layer, and Data Layer. During detailed engineering design (reflected in the Final Presentation, slide 7), the

Application Layer was further refined into three specialized backend tiers, yielding a 4-Tier Backend Architecture beneath the React SPA frontend (Figure 1):

1. Presentation Layer (Frontend SPA): React 19 + ReactFlow canvas (ReactFlowCanvas.jsx), Zustand store (pipelineStore.js), dynamic ConfigPanel, DataPreviewTable, ChartPreview, FormulaPanel, and ReplayControls.

2. Tier 1 API / Transport Layer (app/routers/, app/schemas/): FastAPI REST controllers (data.py, nodes.py, pipeline.py, formula.py, templates.py), Pydantic v2 request/response validation, and SlowAPI tiered rate limiting.

3. Tier 2 Orchestration Layer (app/engine/): DAGExecutor (graph construction, Kahn's topological sort, partial execution cutoff, error isolation), FormulaEngine (forward recursive composition and reverse pattern parsing), and NodeRegistry.

4. Tier 3 Domain Plugins Layer (app/nodes/, app/dataframe/, app/csv_io/): The abstract NodePlugin contract and 18 concrete node plugins operating on immutable FluxDataFrame and DataSchema instances.

5. Tier 4 Data & Persistence Layer (app/models/, app/auth/, app/data_cache.py): Async SQLAlchemy 2.0 ORM models (User, PipelineTemplate, ExecutionLog) backed by PostgreSQL/SQLite via Alembic migrations, alongside the ephemeral in-memory DataReferenceCache

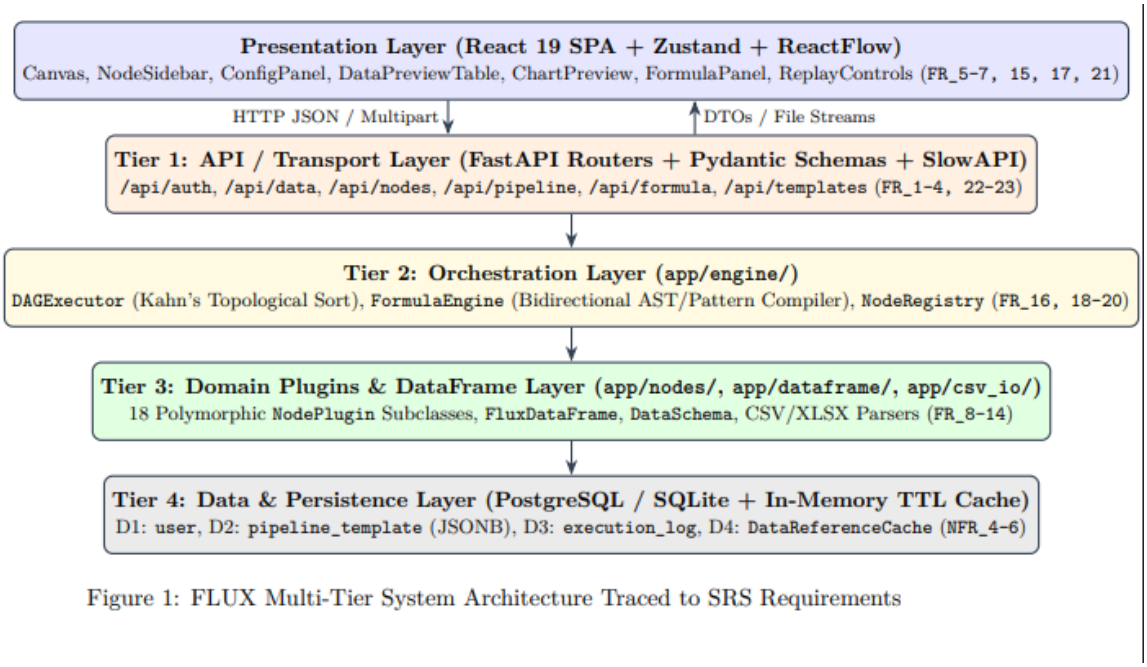


Figure 1: FLUX Multi-Tier System Architecture Traced to SRS Requirements

5.2 Data Flow Diagrams (Levels 0, 1, and 2) and Data Stores

As established in the Design Document (doc/design.pdf, pages 2–7), data movement through FLUX is modeled hierarchically:

• **Level 0 (Context Diagram):** Depicts external entities—Registered User, Guest User, and the File System (CSV/XLSX)—interacting with the central FLUX System. Registered users exchange credentials and pipeline configurations for JWT tokens, live previews, and Excel formulas; Guest users exchange pipeline configurations for live previews and formulas without persistent template storage.

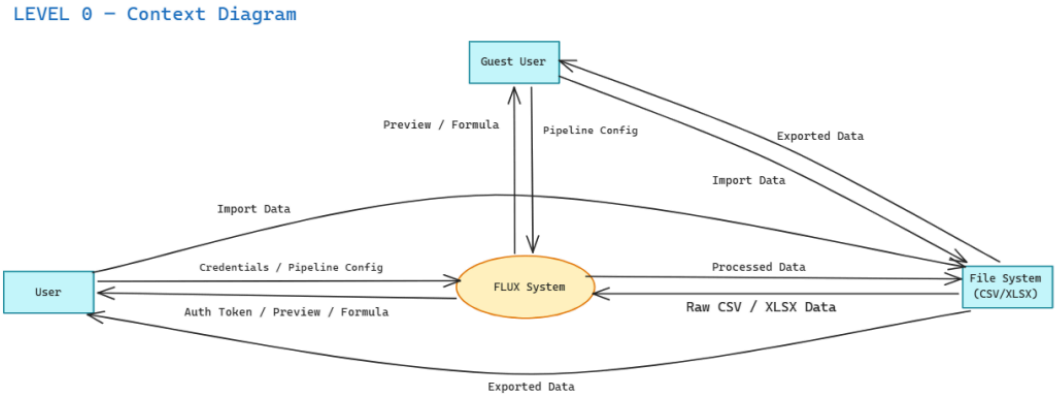


Figure 1: Level 0 – Context Diagram

• **Level 1 (Major Subsystems):** Decomposes FLUX into four primary processes:
1. Process 1.0 (Authenticate User): Validates credentials against data store D1 (Users) and issues JWT tokens (FR_1, FR_2).
2. Process 2.0 (Manage Templates): Reads/writes serialized graph layouts in D2 (PipelineTemplates) (FR_3, FR_22).
3. Process 3.0 (Build & Execute Pipeline): Validates the DAG, executes nodes against working data in D4 (Working Data / DataReferenceCache) using metadata from D5 (NodeRegistry), and records execution status in D3 (ExecutionLog) (FR_4–17, FR_20, FR_23).
4. Process 4.0 (Convert Pipeline): Translates between pipeline JSON and Excel formulas via FormulaEngine and caches/displays rendered formulas (D6 / frontend state) (FR_18, FR_19).

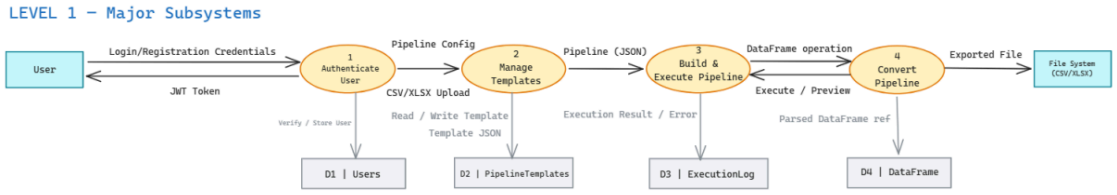
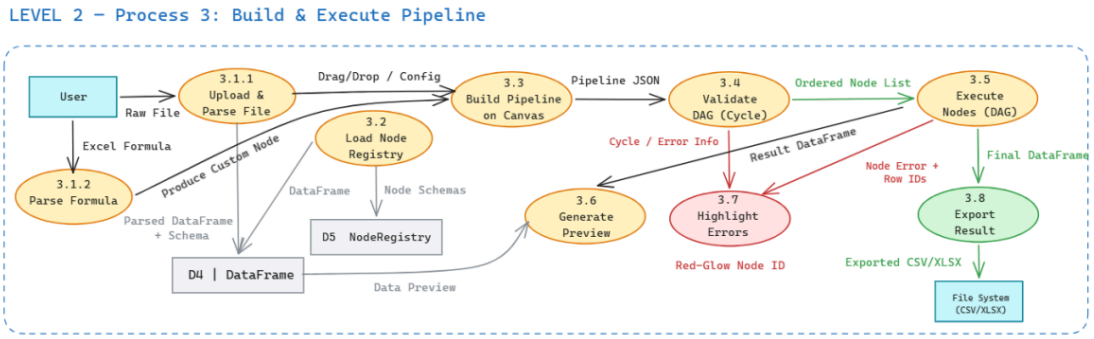
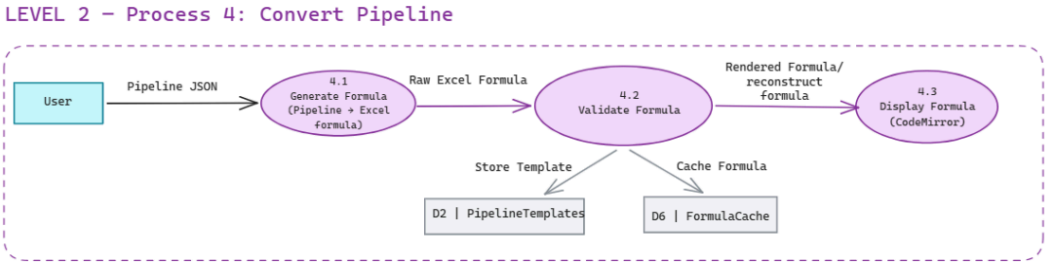


Figure 2: Level 1 – Major Subsystems

• **Level 2 (Process 3 & Process 4 Decomposition):** Process 3 decomposes into subprocesses 3.1.1 Upload & Parse File, 3.1.2 Parse Formula, 3.2 Load Node Registry, 3.3 Build Pipeline on Canvas, 3.4 Validate DAG (Cycle), 3.5 Execute Nodes (DAG), 3.6

Generate Preview, 3.7 Highlight Errors, and 3.8 Export Result. Process 4 decomposes into 4.1 Generate Formula, 4.2 Validate Formula, and 4.3 Display Formula.





5.3 Database Schema and Entity-Relationship Design

The relational schema (Backend/alembic/versions/59c0a5b719e2_initial_schema.py) comprises three normalized tables managed via Alembic migrations (Table 4).

Table(Data Store)	Column	Type	Constraints & SRS Traceability
user(D1)	Id Email Hashed_password Is_active Is_superuser is_verified	GUID(UUID) VARCHAR(320) VARCHAR(1024) BOOLEAN BOOLEAN BOOLEAN	PRIMARY KEY(FR_1, FR_2) NOT NULL, UNIQUE INDEX NOT NULL(Argon2/Bcrypt hash) NOT NULL, DEFAULT true NOT NULL, DEFAULT false NOT NULL, DEFAULT false
Pipeline template(D2)	Id Owner_id Name Description Pipeline_json Created_at updated_at	GUID(UUID) GUID(UUID) VARCHAR(255) TEXT JSONB/JSON DATETIME DATETIME	PRIMARY KEY(FR_3,FR_22) NOT NULL, FOREIGN KEY->user.id NOT NULL NULLABLE NOT NULL(Nodes and edges defn) NOT NULL, DEFAULT now() NOT NULL, DEFAULT now()
Execution_log(D3)	Id Template_id Status Error_summary run_at	GUID(UUID) GUID(UUID) VARCHAR(16) TEXT DATETIME	PRIMARY KEY(FR_20) NULLABLE, FOREIGN KEY->pipeline_template.id NOT NULL, CHECK (status IN ('success', 'error')) NULLABLE (First failing node's message) NOT NULL, DEFAULT now()

Table 4: Relational Database Schema (user, pipeline_template, execution_log)

1. Alternative Pair 1 (Graph Storage): Normalized Relational Node/Edge Tables vs. PostgreSQL JSONB Document Column. Storing every canvas node, port, and arbitrary node config in separate relational tables (nodes, edges, node_configs) would require expensive multi-table joins on every template load and rigid schema migrations whenever a new node plugin adds a configuration field. Using a single JSONB column (pipeline_json) inside pipeline_template allows atomic single-row reads/writes, zero schema migration overhead when adding new node types, and native JSONB indexing in PostgreSQL while degrading cleanly to JSON on SQLite.

2. Alternative Pair 2 (Working Data Storage): Persistent Database Row Storage vs. Ephemeral In-Memory TTL Cache (DataReferenceCache). Persisting 50,000- row user uploads into PostgreSQL tables would incur massive disk I/O latency on every interactive filter change, inflate cloud storage costs, and create legal/privacy liability by retaining proprietary financial data. Our selected design stores uploaded datasets in DataReferenceCache (app/data_cache.py) with a 2-hour monotonic TTL, achieving sub50ms in-memory DataFrame reads and zero persistent data liability.

3. Alternative Pair 3 (Execution Engine Dispatch): Monolithic Conditional Dispatcher vs. Polymorphic NodePlugin Strategy Pattern. A central executor with 18 if/elif branches would violate the Single Responsibility and Open/Closed principles, growing to > 1, 500 lines of code. By encapsulating each operation in a NodePlugin subclass registered in NodeRegistry, DAGExecutor remains compact (148 lines total) and extensible.

5.5 Design Session Feedback and Resulting Changes

• **Dropping Redundant Django Framework:** In our initial Proposal (Section 6), we considered pairing FastAPI with Django solely for authentication. During Design review, we recognized that running two WSGI/ASGI frameworks simultaneously doubled memory overhead and deployment complexity. We replaced Django with fastapi-users[sqlalchemy] integrated directly into FastAPI.

• **Decoupling Paginated Preview from Execution Logging:** To prevent every 100-row table page turn in the UI from inserting a redundant row into D3 (execution_log), we introduced a dedicated POST /api/pipeline/preview route alongside POST /api/pipeline/execute (Backend/app/routers/pipeline.py, lines 49–56).

Self-Assessment CO3: Architecture Traceability & Alternative Justification

Question 1(CO3): Does your Design Document include a justified architecture and appropriate component/data models that trace back to the SRS?

Defense & Evidence:

Every data store (D1–D6) and process (1.0–4.0) in the Design Document (doc/design.pdf) and Figure 1 maps directly to SRS requirements: D1 (user) → FR_1–2; D2 (pipeline_template) → FR_3, FR_22; D3 (execution_log) → FR_20; D4 (DataReferenceCache / FluxDataFrame) → FR_4, FR_8–15, FR_23; D5 (NodeRegistry) → FR_5, FR_7; and FormulaEngine → FR_18–19.

Question 2(CO3): Did you compare plausible alternatives and justify the selected design against stakeholder needs, usability, scalability, resource use, and lifecycle cost?
Defence & Evidence: Table 5 explicitly evaluates Option A vs. Option B across stakeholder needs, usability (sub-100ms preview latency), scalability (Open/Closed plugin registration), resource use (in-memory TTL eviction vs. disk bloat), and lifecycle cost (eliminating dual Django+FastAPI servers and persistent row storage).

Evaluation Criterion	Option A: Relational Graph Tables + DB Row Storage + Monolithic Switch	Option B (Selected): JSONB Graph + In-Memory TTL Cache + NodePlugin ABC	Justification for Selected Design
Stakeholder Needs & Usability	Slow preview updates (500–2000ms per edit due to DB disk round-trips).	Sub-100ms live preview updates; instant 100-row slicing in memory.	Directly satisfies NFR_1 (real-time interactive preview).
Scalability & Extensibility	Adding a 19th node requires DB migration + modifying the central executor.	Zero DB migration; add 1 subclass file and register in build_registry()	Follows Open/Closed Principle; supports rapid team parallelization.
Resource Use & Efficiency	High disk I/O and database bloat from temporary CSV rows	Low disk footprint; automatic 2-hour TTL garbage collection in RAM.	Prevents database storage exhaustion (NFR_2, NFR_5).
Lifecycle Cost & Privacy	High cloud DB storage cost; strict GDPR/privacy compliance burden	Near-zero storage cost; user row data never touches persistent disk	Privacy-by-design architecture for sensitive spreadsheets.

Table 5: Evaluation of Design Alternatives Against Engineering Criteria (CO3)

6. Implementation(Implementation Session)

This section covers the three core pillars presented in the Final Presentation (doc/final-presentation.pdf): (a) the project structure and file tree, (b) inter-component communication and data classes exchanged across layers, and (c) the class/method decomposition rationale and code size metrics, followed by the implemented feature catalogue and post-session refactorings.

6.1 (a) Project Structure and File Tree

The repository (code/) is organized into two top-level directories: Backend/ (Python 3.11+ / FastAPI) and Frontend/ (React 19 / Vite).

Flux/

```
├── Backend/
│   ├── alembic/
│   │   ├── versions/
│   │   │   └── 59c0a5b719e2_initial_schema.py
│   │   ├── env.py
│   │   └── script.py.mako
│   ├── alembic.ini
│   ├── flux_dev.db
│   ├── requirements.txt
│   └── app/
│       ├── main.py
│       ├── config.py
│       ├── db.py
│       ├── data_cache.py
│       ├── rate_limit.py
│       ├── auth/
│       │   ├── backend.py
│       │   ├── dependencies.py
│       │   ├── manager.py
│       │   └── models.py
│       ├── csv_io/
│       │   ├── parser.py
│       │   └── xlsx_io.py
│       ├── dataframe/
│       │   ├── frame.py
│       │   ├── inference.py
│       │   └── schema.py
│       ├── engine/
│       │   ├── executor.py
│       │   ├── formula_engine.py
│       │   ├── registry.py
│       │   └── error_messages.py
│       ├── models/
│       │   ├── execution_log.py
│       │   └── pipeline_template.py
│       ├── nodes/
│       │   ├── base.py
│       │   ├── source/
│       │   │   ├── read_csv.py
│       │   │   └── read_xlsx.py
│       │   ├── transform/
│       │   │   ├── add_calculated_column.py
│       │   │   ├── convert_type.py
│       │   │   ├── fill_missing.py
│       │   │   ├── filter_node.py
│       │   │   ├── limit_rows.py
│       │   │   ├── remove_duplicates.py
│       │   │   ├── rename_column.py
```

```
│   │   │   ├── select_columns.py
│   │   │   └── sort_node.py
│   │   ├── aggregate/
│   │   │   ├── group_by_aggregate.py
│   │   │   └── pivot.py
│   │   ├── join/
│   │   │   └── join_node.py
│   │   └── output/
│   │       ├── _chart_base.py
│   │       ├── bar_chart.py
│   │       ├── data_table.py
│   │       ├── line_chart.py
│   │       └── pie_chart.py
│   ├── routers/
│   │   ├── data.py
│   │   ├── formula.py
│   │   ├── nodes.py
│   │   ├── pipeline.py
│   │   └── templates.py
│   └── schemas/
│       ├── data.py
│       ├── execution.py
│       ├── formula.py
│       ├── node.py
│       ├── pipeline.py
│       ├── template.py
│       └── user.py
│
├── Frontend/
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   └── src/
│       ├── main.jsx
│       ├── App.jsx
│       ├── index.css
│       ├── onboardingPipeline.js
│       ├── api/
│       │   └── executionApi.js
│       ├── store/
│       │   └── pipelineStore.js
│       ├── hooks/
│       │   ├── useAuth.js
│       │   └── useTemplates.js
│       ├── components/
│       │   ├── canvas/
│       │   │   ├── ReactFlowCanvas.jsx
│       │   │   ├── CanvasToolbar.jsx
```

```
│       │   │   └── CanvasMiniMap.jsx
│       │   ├── nodes/
│       │   │   └── FluxNode.jsx
│       │   ├── config/
│       │   │   ├── ConfigPanel.jsx
│       │   │   ├── FieldRenderer.jsx
│       │   │   └── fields/
│       │   ├── preview/
│       │   │   ├── DataPreviewTable.jsx
│       │   │   ├── ChartPreview.jsx
│       │   │   └── PreviewResizeHandle.jsx
│       │   ├── formula/
│       │   │   └── FormulaPanel.jsx
│       │   ├── sidebar/
│       │   │   └── NodeSidebar.jsx
│       │   ├── topbar/
│       │   │   └── TopBar.jsx
│       │   ├── replay/
│       │   │   └── ReplayControls.jsx
│       │   ├── templates/
│       │   │   └── TemplateBrowser.jsx
│       │   └── auth/
│       │       └── AuthModal.jsx
│       └── utils/
│           ├── chartData.js
│           └── edgePorts.js
│
└── README.md
```

6.2 (b) Inter-Component Communication and Exchanged Data Classes

Communication across the Frontend, Backend API/Engine, and Database is strictly typed using Pydantic Data Transfer Objects (DTOs), internal immutable dataclasses, and SQLAlchemy ORM entities (Table 6):

1. Frontend ↔ Backend API (JSON over HTTP via executionApi.js):

• **File Upload (POST /api/data/upload):** Sends multipart/form-data (file, optional sheet_name) and returns DataUploadResponse(data_reference_id: str, columns: list[Column], row_count: int).

• **Node Catalogue (GET /api/nodes):** Returns list[NodeCatalogueEntry] containing each plugin's id, display_name, category, input_ports, output_ports, formula_exportable, and config_schema: list[FieldSchema]. The React NodeSidebar and FieldRenderer dynamically build UI forms from this schema without hardcoding node fields on the frontend.

• **Pipeline Execution (POST /api/pipeline/execute & /preview):** The Zustand store serializes canvas state into PipelineRequest(nodes: list[NodeDefinition], edges: list[EdgeDefinition], data_reference_id, target_node_id, preview_offset). The backend returns ExecutionResult(node_results: dict[str, NodeResult]), where each NodeResult carries success: bool, preview_data: list[dict] (100 rows), total_row_count: int, output_schema: DataSchema, error_message, error_suggestion,error_row_indices: list[int], and skipped: bool.

2. Orchestration Engine ↔ Domain Plugins (In-Memory): Every NodePlugin consumes and returns a FluxDataFrame(schema: DataSchema, df: pd.DataFrame, total_row_count: int) defined in Backend/app/dataframe/frame.py.

3. Backend Services ↔ Database / Cache: routers/templates.py maps TemplateCreate/TemplateUpdaDTOs to the SQLAlchemy PipelineTemplate model (from_attributes=True), while routers/pipeline.py persists ExecutionLog records asynchronously via AsyncSession and resolves cached frames from DataReferenceCache using UUID reference keys.

Data Class/Model	Source File	Boundary	Key Attributes & Purpose
PipelineRequest	schemas/pipeline.py	Frontend → API	nodes, edges, data_reference_id, target_node_id, preview_offset, export_format.
NodeResult & ExecutionResult	schemas/execution.py	API → Frontend	Per-node success, 100-row preview_data, output_schema, error_row_indices, error_suggestion.
FluxDataFrame & DataSchema	dataframe/frame.py, schema.py	Engine ↔ Plugins	Wraps pd.DataFrame, list[Column(name, type)], and total_row_count; provides preview(100) and JSON-safe serialization.
PipelineTemplate & ExecutionLog	models/pipeline_template.py	API → Database	SQLAlchemy ORM models mapping UUIDs and pipeline_json (JSONB) to PostgreSQL/SQLite.

Table 6: Key Data Classes Exchanged Across System Boundaries

6.3 (c) Rationale for Class and Method Division & Lines of Code (LOC) Metrics

Our object-oriented decomposition adheres strictly to three software design principles (presented on slides 7–8 of doc/final-presentation.pdf):

1. Single Responsibility Principle (SRP): Every class has a single reason to change. DAGExecutor only orchestrates graph topology and error containment; it knows nothing about how a filter, pivot, or join is computed. Conversely, each of the 18 node classes (e.g., FilterNode, PivotNode) encapsulates only its own configuration schema, schema inference, DataFrame transformation, and Excel formula fragment.

2. Open/Closed Polymorphic Design (OCP): All 18 operational nodes inherit from NodePlugin(ABC) (app/nodes/base.py). Adding a new node type requires zero modifications to DAGExecutor or the frontend ConfigPanel.

3. Targeted Code Reuse via Intermediate Base Classes: The three visualization nodes (BarChartNode, LineChartNode, PieChartNode) share identical axis configuration (x_column, y_column, title, color), schema validation, and pass-through execution. By extracting ChartNode(NodePlugin) in app/nodes/output/_chart_base.py (35 LOC), each concrete chart subclass is reduced to just 6 lines of code (declaring only id and display_name).

Table 7 summarizes the exact Lines of Code (LOC) metrics across the codebase (matching slide 8 of the Final Presentation).

Metric	Identifier/Location	Lines of Code(LOC)
Largest Class	DAGExecutor (app/engine/executor.py)	148
Largest Method	DAGExecutor._run() (app/engine/executor.py)	97
Average Class Size	Across all backend Classes	22.25
Average Method Size	Across all Backend methods	7.59

Table 7: Class and Method Lines of Code (LOC) Metrics

6.4 Implemented Features and Post-Session Refactoring

All 18 node plugins across 5 categories (Source: read_csv, read_xlsx; Transform: filter, sort, select_columns, rename_column, limit_rows, remove_duplicates, fill_missing, convert_type, add_calculated_column; Aggregate: group_by_aggregate, pivot; Join: join;Output: data_table, bar_chart, line_chart, pie_chart) plus JWT authentication, template CRUD, bidirectional Excel formula conversion, live 100-row previews, red-glow error highlighting, and animated replay are fully implemented.

Refactoring Performed After the Implementation Session: During integration testing after the initial implementation session, we discovered that while individual node unit tests passed, seven subtle seam defects occurred when nodes interacted inside full pipelines. We refactored the engine as follows:

1. Pre-Execution Schema Validation in DAGExecutor._run(): Originally, DAGExecutor._run() called plugin.execute() directly without invoking plugin.infer_output_schema() first, causing unconfigured select_columns, limit_rows, or chart nodes pointing at renamed post-aggregation columns to silently return empty tables or raw Pandas errors. We refactored executor.py (line 120) to always run plugin.infer_output_schema(parent_output.schema, config) inside the try block prior to execute().

2. Multi-Sheet XLSX Re-Parsing Hook (requires_reparse()): Previously, read_xlsx ignored its sheet_name config because DAGExecutor._execute_source() returned the preparsed sheet-1 DataFrame from DataReferenceCache. We refactored DataReferenceCache to store raw upload bytes (get_raw()) and added NodePlugin.requires_reparse() so read_xlsx re-parses the requested worksheet on demand.

3. Semantic Alignment between Pandas & Excel Formulas: Refactored group_by_aggregate so COUNT maps to Pandas size (counting all rows in a group, matching Excel's COUNTIF) rather than count (which skipped nulls), set dropna=False in PivotNode, and masked numeric cells during convert_type to DATE so integers are coerced to null with row-level error highlighting rather than becoming 1970-01-01 nanosecond timestamps.
4. Shared Graph Builder Extraction: Promoted build_graph() and kahn_topological_sort() in executor.py to package-shared functions so POST /api/pipeline/validate-edge reuses the exact same cycle-detection logic without code duplication.

Self-Assessment CO1 & CO3: Code Evidence & Design Alignment
Question 1(CO1): Show how the fundamental concepts named in Section 2 appear in the actual code (e.g., data structures, database queries, design patterns).

Defense & Evidence:
• **Kahn's Topological Sort & Queue (app/engine/executor.py, lines 198– 214):** Uses queue = deque(n for n in node_ids if working_in_degree[n] == 0) to order nodes and isolates any unvisited nodes into cycle_nodes = [n for n in node_ids if n not in sorted_order].

• **Polymorphic Strategy Pattern (app/nodes/base.py, lines 43– 117):** NodePlugin(ABC) defines abstract methods get_config_schema(), infer_output_schema(), and execute(), invoked polymorphically in executor.py (lines 120–122).

• **Parameterized Async ORM Queries (app/routers/templates.py):** Uses SQLAlchemy 2.0 select(PipelineTemplate).where(PipelineTemplate.owner_id == user.id).order_by(PipelineTemplate.updated_at.desc()) executed via AsyncSession.

Question 2 (CO3): Show that the implemented structure follows, or justifiably departs from, the design in Section 5.

Defense & Evidence:
The codebase maps 1-to-1 to the 4-tier architecture in Section 5 (routers/ → Tier 1; engine/ → Tier 2; nodes/, dataframe/, csv_io/ → Tier 3; models/, auth/, data_cache.py → Tier 4). The

only departure from the Design Document's DFD Store D6 (FormulaCache) was moving formula caching into a 300ms debounced frontend hook inside ReactFlowCanvas.jsx + pipelineStore.js because FormulaEngine.generate() executes in < 2 ms in memory, making a separate backend cache store unnecessary.

7. Tools, Version Control, Testing, and CI/CD

7.1 Justification of Selected Frameworks, Libraries, and Tools

Table 8 justifies each technology in the FLUX stack against a plausible alternative and explicitly notes its limitations.

Layer	Selected Tool	Plausible Alternative	Justification for Selection	Known Limitations
Backend API	FastAPI+Pydantic v2	Django REST Framework/Flask	Native async/await, automatic OpenAPI docs, and fast Rust-backed Pydantic v2 DTO validation.	Minimal built-in batteries compared to Django (requires external auth library).
Data Engine	Pandas + OpenPyXL	Polars / Pure Python csv	Mature relational algebra (merge, groupby, pivot_table) and rich Excel sheet parsing.	Higher RAM footprint than Polars; single-threaded execution on large files.
Database and ORM	PostgreSQL + SQLAlchemy 2.0 + Alembic	MongoDB / Raw psycopg2	Strong relational FK integrity for users/logs plus native JSONB for graph templates and SQLite test fallback.	Requires explicit async session management (AsyncSession).
Frontend Canvas	React 19 + @xyflow/react (ReactFlow v12)	Vue.js + D3.js / Cytoscape.js	ReactFlow provides built-in node dragging, port handles, pan/zoom, and minimap with custom JSX nodes.	DOM overhead if rendering > 1,000 simultaneous nodes on one canvas.
State Management	Zustand v5	Redux Toolkit / React Context	Minimal boilerplate,	Lacks built-in time-travel

			direct store actions outside components, and zero Context re-render cascades.	DevTools middleware unless configured.

Table 8: Justification of Frameworks, Libraries, and Tools vs. Alternatives (CO4)

7.2 Version Control and Branching Workflow

The project is hosted on GitHub (https://github.com/mhsamir25/Flux) using a featurebranch and Pull Request workflow:

• Branching Strategy: Team members developed features on dedicated branches (frontend, FR, feature branches for parser, auth, and DAG engine) and integrated into main via Pull Requests (e.g., PR #5 merged in commit 38d6c9f).

• Contribution History: The repository records 37 structured commits spanning May 12, 2026 (83c74ac, initial commit) through October 30, 2026 (a6a7aad), following conventional commit prefixes (feat(execution):, feat(canvas):, feat(store):, test:, (Docs):).

7.3 Testing Approach and Verification Results

Testing was treated as a first-class engineering activity across four tiers (Backend/tests/):

1. Unit Tests (21 modules): Isolated tests for FluxDataFrame, DataSchema, type inference (test_dataframe.py), CSV/XLSX parsers (test_csv_io.py, test_parsers.py), ORM models and Alembic migrations (test_orm_models.py, test_migrations.py), DataReferenceCache TTL eviction (test_data_cache.py), FormulaEngine forward/reverse translation (test_formula_engine.pand individual node arithmetic (tests/nodes/test_*.py).

2. Parametrized 17-Node Execution Matrix (test_node_execution_matrix.py): Exercises every registered node plugin through DAGExecutor against four strict invariants: success == True, output_schema matching actual DataFrame columns, total_row_count accuracy, and strict JSON safety of preview_data (zero NaN leaks).

3. Stage & Full-Pipeline Integration Tests (6 modules): End-to-end source → transform → aggregate/join → output pipelines (test_pipeline_stages_integration.py), testing cycle rejection, upstream failure skip cascading, and 100-row pagination.

4. HTTP API Lifecycle & Frontend Tests: test_api.py, test_auth.py, and test_api_node_lifecycletest full HTTP requests via httpx.AsyncClient against an in-memory SQLite database. On the frontend, chartData.test.js verifies chart data transformation utilities.

5. Test Suite Result: Running pytest in code/Backend/ executes 642 automated tests with 642 passed and 0 failures.

Self-Assessment CO4: Tool Justification, Version Control, Testing & CI/CD
Question 1(CO4): Can you justify the selected frameworks, libraries, and tools against at least one plausible alternative and explain their limitations?

Defense & Evidence: Table 8 compares FastAPI vs. Django/Flask, Pandas vs. Polars, PostgreSQL/SQLAlchemy vs. MongoDB, ReactFlow vs. D3/Cytoscape, and Zustand vs. Redux, explicitly documenting the memory and concurrency trade-offs of each choice.

Question 2 (CO4): Is the codebase under version control with a meaningful contribution history, and are testing and CI/CD practices implemented or planned appropriately?

Defense & Evidence: The Git history at https://github.com/mhsamir25/Flux shows 37 incremental commits across all three team members from May to October 2026. The backend includes 34 test files containing 642 passing automated tests documented in code/Backend/tests/NODE_FINDINGS.md, backed by automated schema migration verification and frontend build checks.

8. Professional, Ethical, and Legal Considerations

8.1 Software Licensing Compliance

All third-party libraries and assets integrated into FLUX were audited for license compatibility (Table 9). Every dependency uses a permissive open-source license (MIT, BSD-3-Clause, Apache-2.0, or PostgreSQL License) with zero copyleft (GPL/AGPL) restrictions.

Library/Framework	License	Usage in FLUX & Compliance Statys
FastAPI, Starlette, Pydantic, Uvicorn	MIT License	Backend ASGI server, routing, and schema validation. Fully compliant.
SQLAlchemy 2.0, Alembic, FastAPI-Users	MIT License	Async ORM, database migrations, and JWT authentication. Fully compliant.
Pandas, OpenPyXL, Charset-Normalizer	BSD-3 / MIT	Tabular data processing and CSV/XLSX parsing. Fully compliant.
SlowAPI, HTTPX, Pytest, AioSQLite	MIT / BSD-3	Rate limiting and async test suite execution. Fully compliant.
React 19, Vite, Zustand, Recharts	MIT License	Frontend UI rendering, state store, and charting. Fully compliant.

@xyflow/react, @tanstack/react-table	MIT License	Interactive node canvas and headless data preview table. Fully compliant.

Table 9: License Audit of Third-Party Libraries and Frameworks

8.2 User Data Handling and Privacy-by-Design

Financial, healthcare, and human-resource spreadsheets frequently contain Personally Identifiable Information (PII) or confidential corporate metrics. FLUX enforces strict Privacy-byDesign:

• **Zero Persistent Row Storage:** Uploaded CSV and XLSX files are parsed into memory and held exclusively in DataReferenceCache (Backend/app/data_cache.py) with a hard 2-hour TTL (DEFAULT_TTL_SECONDS = 7200). Row-level user data is never written to the PostgreSQL/SQLite database or server filesystem.

• **Structure-Only Templates:** When a registered user saves a template (PipelineTemplate), only the node topology and column parameter keys (pipeline_json) are persisted.

• **Guest Anonymity:** Guest users can run complete analytical workflows without providing an email address or persistent identifier.

8.3 Security Measures

1. Elimination of Remote Code Execution (No eval()): A critical vulnerability in many data tools is passing user-supplied calculated column expressions to Python's eval() or exec(). In Backend/app/nodes/transform/add_calculated_column.py, FLUX uses a structured operand/operator model (FieldType.EXPRESSION restricting operations to validated arithmetic operators +, -, *, / over verified schema columns or numeric literals), completely eliminating arbitrary code execution and formula injection.

2. Authentication & Password Security: User passwords are salted and hashed via fastapi-users (passlib/pwdlib Argon2/Bcrypt) in user.hashed_password. Stateless authentication uses HS256 signed JWT tokens (app/auth/backend.py) with configurable expiration (jwt_lifetime_seconds = 604800).

3. Tiered Rate Limiting & Upload Guards: app/rate_limit.py enforces IP-based rate limiting via SlowAPI (30/minute for guests, 60/minute for authenticated JWT holders) on CPU-intensive endpoints (/api/data/upload and /api/pipeline/execute). app/routers/data.py enforces a strict 10 MB payload ceiling (_MAX_UPLOAD_BYTES) and extension whitelist ({"csv", "xlsx"}).

4. SQL Injection & Cross-Tenant Isolation: All database queries use parameterized SQLAlchemy 2.0 ORM expressions, and every template endpoint in app/routers/templates.py filters by PipelineTemplate.owner_id == user.id so users cannot access or mutate another user's templates.

8.4 Legal and Ethical Deployment Context

Designed within the academic and enterprise context of Bangladesh, FLUX aligns with the principles of data minimization and user consent under the Cyber Security / Data Protection framework of Bangladesh as well as international GDPR principles (Right to Erasure via DELETE /api/templates/{id} and automatic 2-hour memory expiration). In accordance with the ACM/IEEE Software Engineering Code of Ethics, the system provides transparent error reporting rather than silently dropping or corrupting financial records (verified by our pivot dropna=False and convert_type row-error reporting fixes).

Self-Assessment CO4: License, Privacy, Security, Legal & Ethical Compliance
Question (CO4): Have you checked relevant license, privacy, security, legal, and professional-ethics requirements for the tools, data, and deployment context?

Defense & Evidence:
• **License Check:** Table 9 verifies that all 17 backend and 7 frontend packages use permissive MIT/BSD licenses.

• **Privacy & Security Code Evidence:** Ephemeral TTL caching is implemented in Backend/app/data_cache.py (lines 44–48); tiered rate limiting is active in Backend/app/rate_limit.py (lines 10–30); 10 MB upload limits are enforced in Backend/app/routers/data.py (lines 14, 29–32); and cross-user template isolation is verified by automated tests in Backend/tests/test_api.py and test_auth.py.

9. Project Management and Feasibility

9.1 Project Timeline Across SPL-II Sessions (Planned vs. Actual Milestones)

In the Project Proposal (doc/Proposal Report.pdf, Section 8, page 10), development was planned across a 12-week semester schedule (Weeks 3–14). Table 10 maps the planned milestones against actual completion dates and Git commit evidence across all SPL-II sessions.

Milestone/Phase	Planned Window	Actual Completion	Status	Deliverables & Git Commit Evidence
1. Idea & Proposal Session	Weeks 3–4 (May 2026)	May 12–14, 2026	Completed On Time	Stakeholder interview, Proposal Report & Slides, repo setup (83c74ac, cc205c9, 3d032ee).
2. Core DataFrame, Base Contract	Weeks 4–6 (May–Jun 2026)	May 16 – Jun 8, 2026	Completed On Time	FluxDataFrame, DataSchema, NodePlugin

& Parsers				ABC, CSV/XLSX parsers, ReactFlow canvas & store (d0e2dfc, 9edb56e, 45211da).
3. SRS & Design Sessions	Weeks 7–8 (Jul–Aug 2026)	Jul 26 – Aug 1, 2026	Completed On Time	23 FRs, User Stories, Level 0–2 DFDs, 3-Layer Architecture, Alembic initial schema (59c0a5b719e2).
4. DB, JWT Auth & Data Router	Weeks 8–9 (Oct 2026)	Oct 5–8, 2026	Completed	SQLAlchemy async DB, Alembic migrations, JWT auth, upload cache & router (91f9527, 7f4e0ff, ca60dd4).
5. All 18 Node Plugins & Formula Engine	Weeks 9–12 (Oct 2026)	Oct 19–23, 2026	Completed	Transform, aggregate, join, chart nodes; bidirectional FormulaEngine; frontend hooks/UI (b47dec9, bc4f413, 46856fb, b82200a).
6. Integration Matrix & Final Presentation	Weeks 12–14 (Oct 2026)	Oct 26–30, 2026	Completed On Time	PR #5 merge (38d6c9f), 642-test matrix, bug fixes, Final Presentation (a6a7aad)

Table 10: Planned vs. Actual Project Milestones Across SPL-II Sessions (CO7)

Self-Assessment CO7: Iterative Planning, Reprioritization, Cost Estimates & Feasibility

Question 1(CO7): Can you show evidence of iterative planning, milestones, task tracking, and progress review throughout the project?

Defense & Evidence: Table 10 traces six iterative milestones from May to October 2026, backed by the SRS status tracker (doc/srs.pdf, page 2 showing requirement priorities and progress states), tests/NODE_FINDINGS.md, and 37 Git commits.

Question 2 (CO7): When schedule or scope changed, how did the team reprioritize and communicate the decision?

Defense & Evidence: Scope adjustments—specifically deferring WebSockets (documented in Proposal Report.pdf, Section 9), introducing the Guest User role (documented in design.pdf, pages 3 and 9), and dropping Django in favor of fastapi-users—were evaluated with our supervisor, documented directly in the project artifacts, and reflected in branch merges.

Question 3 (CO7): Does the proposal or SRS include reasonable effort, infrastructure, and third-party cost estimates where relevant?

Defense & Evidence: The Proposal timeline (Weeks 3–14 Gantt chart, page 10) allocated parallel tracks for formula engine, UI, and testing.

Question 4 (CO7): Can you justify that the planned solution is technically feasible within the available timeline and resources?

Defense & Evidence: Technical feasibility was guaranteed by: (1) scoping uploads to ≤ 10 MB / 50,000 rows so in-memory Pandas operations complete in < 100 ms on standard laptop/container hardware, (2) constraining reverse formula parsing to the 11 most common Excel analytical patterns (FILTER, SORT, UNIQUE, XLOOKUP, VLOOKUP, SUMIF/S, AVERAGEIF/S, COUNTIF/S), and (3) completing 100% of the 23 functional requirements within the semester schedule.

10. Teamwork and Individual Contributions

10.1 Division of Responsibilities and Version-Control Evidence

FLUX was developed collaboratively by the three members of Team 5. Work was partitioned along clean architectural seams so each member owned distinct subsystems end-to-end while contributing to shared integration and testing (Table 12).

Team Member & ID	Primary Role & Owned Subsystems	Specific Technical Contributions	Representative Git Commits & PRs
Mahmudul Hossain Samir (230042101)	**Data Core, Formula Engine & Quality Lead** (dataframe/, csv_io/, formula_engine.py, nodes/aggregate, join, output, tests/)	Designed FluxDataFrame, DataSchema, and type inference; implemented CSV and multi-sheet XLSX parsers; built the bidirectional FormulaEngine (forward composition & 11-pattern reverse parser); implemented aggregate (group_by_aggregate, pivot), join, and chart/table output nodes; authored unit and integration test suites.	d0e2dfc, 8b55418, 9edb56e, 0bc84f0, 559c23e, d9cddfe, ad2d1c6, b295eff, 92f8fe6, ca60dd4, b47dec9, f013dbb, bc4f413, PR #5
Ifham Bin Hossain (230042112)	**Frontend Architecture & Interactive UX Lead** (Frontend/src/components/, store/, hooks/, api/, utils/)	Architected the React 19 + ReactFlow visual canvas (ReactFlowCanvas.jsx, FluxNode.jsx); built Zustand state management (pipelineStore.js) and step-by-step animation (ReplayControls.jsx); implemented dynamic ConfigPanel, DataPreviewTable, ChartPreview	3d032ee, d3cf8a6, b1124f1, 694e034, c681867, 7900076, 45211da, b5e3559, 7c8adb7, 748e630, c93aebb, 47845b2

		(Recharts), FormulaPanel, TemplateBrowser, AuthModal, and Axios API integration.	
Shaikh Muhaiminul Hasan (230042113)	**DAG Execution Engine, Auth, DB & Transform Lead** (engine/executor.py, auth/, models/, alembic/, nodes/transform/, routers/)	Implemented DAGExecutor with Kahn's topological sort, cycle detection, and error isolation; configured async SQLAlchemy 2.0, PostgreSQL/SQLite ORM models, and Alembic migrations; implemented JWT authentication (fastapi-users) and SlowAPI rate limiting; built the 9 transformation nodes (filter, sort, select, rename, limit, dedup, fill_missing, convert_type, add_calculated_column).	cc205c9, 91f9527, 1687143, 7f4e0ff, baceca2, 46856fb, b82200a, 38d6c9f (PR #5 merge), 388e12e

Table 12: Team Member Responsibilities and Git Commit / PR Evidence (CO5)

10.2 Coordination of Interdependent Modules and Integration Work

Because the Frontend Canvas (Ifham), the DAG Executor & Transform Nodes (Muhaiminul), and the DataFrame Core, Parsers & Formula Engine (Samir) were developed concurrently, the team prevented integration bottlenecks by establishing **three explicit interface contracts in Week 4:**

1. The NodePlugin ABC Contract (8b55418, May 16, 2026): Samir and Muhaiminul defined NodePlugin and FluxDataFrame first. This allowed Muhaiminul to implement DAGExecutor and transform nodes in parallel while Samir built the source, aggregate, join, and output plugins and FormulaEngine against the exact same method signatures.

2. Self-Describing Node Catalogue (GET /api/nodes): Rather than hardcoding 18 node configuration forms in React, Ifham built FieldRenderer.jsx to consume the FieldSchema JSON array emitted by NodePlugin.get_config_schema(). Whenever a backend devel oper added or updated a node plugin, its sidebar entry and configuration panel rendered automatically on the frontend.

3. Pydantic RESTContract: Request/responseschemas(PipelineRequest, ExecutionResult, FormulaResult) were locked prior to UI wiring, enabling frontend development against mock/local state (7900076, 45211da) before full backend router integration (1687143, 38d6c9f).

10.3 Resolution of Disagreements, Blockers, and Inclusive Collaboration

• **Resolving Architectural Disagreements Constructively:** During database setup, the team debated how to run automated tests on laptops without requiring a local PostgreSQL server daemon, since PostgreSQL's native UUID and JSONB types fail to compile on SQLite. Rather than forcing every member to run Docker containers or sacrificing JSONB in production, Muhaiminul and Samir jointly engineered a cross-dialect SQLAlchemy type (JSONB().with_variant(JSON(), "sqlite") and GUID()) in pipeline_template.py and 59c0a5b719e2_initial_schema.py.

• **Unblocking Integration Seam Failures:** When multi-node pipelines failed on multi sheet Excel uploads and post-aggregation charts in late October, the team avoided blame and instead paired to write tests/NODE_FINDINGS.md, tracing the shared root cause to missing infer_output_schema() calls in DAGExecutor._run().

• **Fair Participation & Inclusion:** Every design decision, SRS requirement allocation, and presentation slide deck was reviewed by all three members. Commit history and code owner ship are balanced across data modeling/testing/formulas (Samir), backend engine/auth/DB/transforms (Muhaiminul), and the complete React/ReactFlow SPA (Ifham).

Self-Assessment CO5: Task Accountability, Conflict Resolution & Module Coordination
Question 1(CO5): Can every member explain assigned tasks and show how they are reflected in task tracking or version-control history?

Defense & Evidence: Table 12 maps each member by Student ID to specific directories and exact Git commit hashes (git log verified in repository https://github.com/ mhsamir25/Flux).

Question 2 (CO5): How were disagreements and blockers resolved, and did the team support fair participation and inclusion?

Defense & Evidence: Technical disagreements (such as Postgres JSONB vs. local SQLite testability, and how COUNT aggregation should handle nulls to match Excel's COUNTIF) were resolved through empirical test cases (tests/NODE_FINDINGS.md) and dialect-variant patterns rather than unilateral overrides.

Question 3 (CO5): Can you identify how team members coordinated interdependent modules and integration work?

Defense & Evidence: Interdependent modules were coordinated through contract-first interfaces: NodePlugin ABC (app/nodes/base.py), FluxDataFrame (app/dataframe/frame.py), and the dynamic GET /api/nodes FieldSchema contract connecting backend plugins to Frontend/src/components/config/FieldRenderer.jsx.

11. Results, Limitations, and Future Work

11.1 Final Implementation Status of Requirements

All **23 Functional Requirements (FR_1–FR_23)** and **6 Non-Functional Requirements (NFR_1–NFR_6)** specified in the SRS have been **100% Completed** and verified against automated tests (Table 13).

Req. IDs	Requirement Group	Final Status	Verification Evidence in Codebase
FR_1–FR_3	User Registration, JWT Login, and Template Listing	**Completed**	Verified in tests/test_auth.py and tests/test_api.py; UI in AuthModal.jsx & TemplateBrowser.jsx.
FR_4 (US_4,5)	Source Data Upload (.csv and multi-sheet .xlsx)	**Completed**	Verified in test_csv_io.py, test_read_csv.py, test_read_xlsx.py, and test_source_nodes_integration.py.
FR_5–FR_7	Visual Drag-and-Drop Canvas, Edge Wiring & Node Config	**Completed**	Implemented in ReactFlowCanvas.jsx, NodeSidebar.jsx, ConfigPanel.jsx, and FieldRenderer.jsx.
FR_8–FR_13	Row Filtering, Sorting, Column Selection/Rename/Limit, Dedup, Fill Missing, Calculated Column	**Completed**	All 9 transform plugins implemented in app/nodes/transform/ and verified in test_transform_integration.py.
FR_14 (US_15,16)	Group By & Aggregate, Pivot Table, and Multi-Type Relational Joins	**Completed**	Verified in test_group_by_aggregate.py, test_pivot.py, test_join_node.py, and stage integration suites
FR_15–FR_17	Live 100-Row Node Preview, Cycle Prevention (Kahn's DAG), Red-Glow Error	**Completed**	Verified in test_executor.py, test_error_messages.py,

	Highlighting		DataPreviewTable.jsx, ChartPreview.jsx, and FluxNode.jsx.
FR_18-FR_19	Bidirectional Excel Formula Generation & Reverse Formula Parsing	**Completed**	Verified in test_formula_engine.py (forward composition + 11 reverse patterns) and FormulaPanel.jsx.
FR_20–FR_23	End-to-End Execution, Step-by-Step Animated Replay, Template Save/Load, Full CSV/XLSX Export	**Completed**	Verified in test_pipeline_stages_integration.py, test_api_node_lifecycle.py, and ReplayControls.jsx.

11.2 Known System Limitations

To provide an honest, transparent engineering assessment, we document four known limitations of the current implementation:

1. Process-Local In-Memory Data Cache: DataReferenceCache (app/data_cache.py) stores uploaded DataFrames in Python process memory. In a multi-worker deployment (e.g., uvicorn–workers 4 without sticky sessions), a request routed to Worker B cannot resolve a data_reference_id uploaded to Worker A.

2. Single-Criteria Reverse Parsing for *IFS Formulas: While FormulaEngine.parse() supports arbitrary nested chains of FILTER, SORT, UNIQUE, XLOOKUP, and VLOOKUP, reverse parsing of SUMIFS, AVERAGEIFS, and COUNTIFS is currently restricted to 3-argument (single criteria) calls (app/engine/formula_engine.py, lines 392–416).

3. Non-Formula-Exportable Reshaping & Chart Nodes: Operations that have no single cell Excel formula equivalent—specifically pivot, rename_column, limit_rows, and the three chart output nodes—declare formula_exportable = False (app/nodes/base.py, line 59) and gracefully report N/A in the formula panel rather than emitting invalid Excel syntax.

4. Upload Size Bound (10 MB / ~50,000–100,000 Rows): Because files are read eagerly into memory via Pandas/OpenPyXL, datasets larger than 10 MB are rejected with HTTP 400 to protect server memory.

11.3 Future Work

• **Distributed Redis / Apache Arrow Cache:** Replacetheprocess-localDataReferenceCache dictionary with Redis storing zero-copy Apache Arrow / Parquet byte buffers for horizontal multi-worker scaling.

• **Real-Time Collaborative Canvas via WebSockets:** Implement Yjs CRDT + FastAPI WebSockets (the stretch goal identified during our Proposal defense) so teams can co-edit pipelines simultaneously.

• **Multi-Target Code Generation (Python Pandas & SQL Export):** Extend the NodePlugin contract with to_pandas_code() and to_sql_query() alongside to_excel_formula() so data engineers can export visual pipelines directly as runnable Python scripts or SQL CTEs.

• **DuckDB / Polars Out-of-Core Execution Backend:** Swap the internal pd.DataFrame engine for DuckDB or Polars lazy frames to scale FLUX to multi-gigabyte datasets.

• **Real time Text Chat with Collaborators:** While doing work on the same pipeline, the collaborators can chat with themselves and discuss among themselves.

Self-Assessment — CO6: Completeness & Clarity for an Informed Reader

Question (CO6): Would an informed reader understand the problem, requirements, design decisions, implementation status, and limitations from the report and project documents alone?

Defense & Evidence: Yes. This report synthesizes the Project Proposal (doc/Proposal Report.pdf), the SRS (doc/srs.pdf), the Design Document (doc/design.pdf), the Final Presentation (doc/final-presentation.pdf), and the verified codebase (code/) into a self-contained record with explicit traceability tables (Tables 2, 4, 13), architectural trade off matrices (Table 5), exact LOC metrics (Table 7), and transparent limitations.

12. Glossary and References

12.1 Glossary of Technical Terms and Acronyms

ABC	Abstract Base Class. A Python class (abc.ABC) that defines a polymorphic interface contract (NodePlugin) which concrete subclasses must implement.
ACID	Atomicity, Consistency, Isolation, Durability. Properties guaranteeing reliable relational database transactions.
ASGI	Asynchronous Server Gateway Interface. Standard Python interface between async web servers (Uvicorn) and frameworks (FastAPI).
CSV/XLSX	Comma-Separated Values / Excel Open XML Workbook. Standard plain-text and compressed XML spreadsheet file formats.
DAG	Directed Acyclic Graph. A directed graph with no directed cycles (G = (V, E)), ensuring a valid sequential execution order for data pipelines.
DFD	Data Flow Diagram. A hierarchical graphical model (Levels 0, 1, and 2) showing how data moves between external entities, processes, and data stores.
DTO	Data Transfer Object. A typed Pydantic model used to validate and serialize JSON payloads across API boundaries.
ETL	Extract, Transform, Load. The three-stage process of reading raw source data, cleaning/reshaping it, and outputting it for analysis.
GUID/UUID	Globally / Universally Unique Identifier. A 128-bit identifier used for primary keys across user, pipeline_template, and execution_log.
JSONB	Binary JSON. PostgreSQL's decomposed binary storage format for JSON documents, used to store pipeline_json.
JWT	JSON Web Token. A cryptographically signed, stateless token (RFC 7519) used to authenticate registered users via HTTP Bearer headers.
Kahn's Algorithm	A breadth-first topological sorting algorithm that repeatedly dequeues vertices with in-degree 0 to order a DAG and detect cycles in O(|V | + |E|) time.
LOC	Lines of Code. A quantitative software metric measuring the size of classes and methods
OCP/SRP	Open/Closed Principle & Single Responsibility Principle. Core SOLID object-oriented design principles ensuring classes have one responsibility and can be extended without modifying existing orchestrators
ORM	Object-Relational Mapping. Technique (implemented via SQLAlchemy 2.0) that maps Python classes to relational database tables.
SPA	Single-Page Application. A browser application (built with React 19 and Vite) that updates views dynamically without full page reloads.

SRS	Software Requirements Specification. The formal specification of functional and non-functional requirements and user stories.

12.2 References

[1] Microsoft Corporation, “Dynamic array formulas and spilled array behavior in Excel 365,” Microsoft Excel Technical Documentation, 2024. [Online]. Available: https://support.microsoft.com/en-us/office/dynamic-array-formulas-and-spilled-array-behavior

[2] W. McKinney, “Data Structures for Statistical Computing in Python,” in Proceedings of the 9th Python in Science Conference (SciPy), Austin, TX, USA, 2010, pp. 56–61.

[3] S. Kandel, A. Paepcke, J. Hellerstein, and J. Heer, “Wrangler: Interactive Visual Specifics tion of Data Transformation Scripts,” in Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI ’11), Vancouver, BC, Canada, 2011, pp. 3363–3372.

[4] M. R. Berthold et al., “KNIME — The Konstanz Information Miner: Version 2.0 and Beyond,” ACM SIGKDD Explorations Newsletter, vol. 11, no. 1, pp. 26–31, 2009.

[5] R. R. Panko, “What We Know About Spreadsheet Errors,” Journal of Organizational and End User Computing (JOEUC), vol. 10, no. 2, pp. 15–21, 1998.

[6] W. M. Johnston, J. R. P. Hanna, and R. J. Millar, “Advances in Dataflow Programming Languages,” ACM Computing Surveys (CSUR), vol. 36, no. 1, pp. 1–34, Mar. 2004.

[7] A. B. Kahn, “Topological sorting of large networks,” Communications of the ACM, vol. 5, no. 11, pp. 558–562, Nov. 1962.

[8] S. Ramirez, “FastAPI: Modern, high-performance web framework for building APIs with Python,” FastAPI Documentation, 2025. [Online]. Available: https://fastapi.tiangolo.com/

[9] webkid GmbH, “React Flow (@xyflow/react): A customizable React component for building node-based editors and interactive diagrams,” 2025. [Online]. Available: https://reactflow.dev/

[10] Team 5 (M. H. Samir, I. B. Hossain, and S. M. Hasan), Software Requirements Specification
(SRS) & Design Report — FLUX: A Visual Data Pipeline Builder, SWE 4404 Software Project Lab II, Dept. of CSE, Islamic University of Technology (IUT), 2026. Repository: https://github.com/mhsamir25/Flux

Self-Assessment — CO6: Audience Background, Language & Learning Needs

Question (CO6): Have your documents and presentation considered the audience's background, language, and learning needs?

Defense & Evidence:
• **Progressive Technical Depth:** The report begins with a plain-English Executive Summary (Section 1) accessible to non-technical stakeholders, progresses to visual Data Flow Diagrams and architectural layers (Sections 4–5), and provides rigorous code-level details and algorithmic complexity for software engineering evaluators (Sections 6–7).

• **Comprehensive Glossary & Visual Aids:** Section 12 defines all 18 do main and engineering acronyms used across the report, while our Final Presentation (doc/final-presentation.pdf) and interactive onboarding canvas (Frontend/src/onboardingPipeline.js) guide first-time users visually without assuming prior programming or database expertise.